SkyAccel: A Cross-Cloud Acceleration System for Latency-Sensitive Workloads

Matthew Yang Liu
Independent Researcher
Hangzhou, China
matthewliu58@gmail.com

Hao Che
University of Texas at Arlington
Arlington, USA
hche@cse.uta.edu

Schahram Dustdar
ICREA
Barcelona, Spain
schahram.dustdar@upf.edu

Abstract

Modern cloud services increasingly rely on latency-sensitive workloads (LSWs) characterized by bursty arrivals, CPU-intensive execution, and strict latency requirements. Existing commercial cloud acceleration services (CCASs) are tightly coupled to vendor-specific infrastructures, limiting deployment flexibility in heterogeneous multi-cloud environments. We present SkyAccel, a cross-cloud traffic acceleration system for LSWs. We observe an asymmetry in end-to-end traffic control: edge execution is dominated by CPU contention and stability constraints, while core routing is governed by dynamic inter-cloud path dynamics. Based on this insight, SkyAccel adopts an asymmetry-aware control decomposition (ACD) that separates execution stability control at the edge from routing optimization in the core. It builds a cloud-based overlay spanning multiple cloud providers to dynamically steer requests and schedule execution across distributed virtual machines (VMs). Evaluation shows that SkyAccel reduces 90th-percentile latency by up to 60.07% across multi-cloud deployments, with gains varying by network conditions, maintains stable performance under high-concurrency workloads on resource-constrained instances, and reduces traffic delivery cost by 4–30× compared to state-of-the-art CCASs.

CCS Concepts

• **Networks** → **Network services**; **Network architectures**; • **Computer systems organization** → Cloud computing.

Keywords

Multi-cloud computing, Latency-sensitive workloads, Traffic acceleration, Overlay network, Network management

1 Introduction

Many modern Internet services, such as video platforms [71] and online services [52], generate network traffic consisting of rich multimedia content [33, 74] and latency-sensitive control traffic. The latter includes coordination signals, authentication requests, and interactive service requests (e.g., AI inference), which are critical to service responsiveness and online functionality. These small-size messages are highly latency-sensitive, where even tens of milliseconds of additional wide-area network (WAN) latency can degrade user-perceived responsiveness in geographically distributed deployments [70]. Moreover, such latency-sensitive traffic exhibits burstiness due to user activity surges, leading to transient queue buildup and increased latency. From a network perspective, inter-region variability further amplifies latency jitter and degrades path-level stability. These observations motivate a cross-cloud acceleration layer that adaptively steers latency-sensitive workloads (LSWs) along stable, low-latency paths under dynamic WAN conditions.

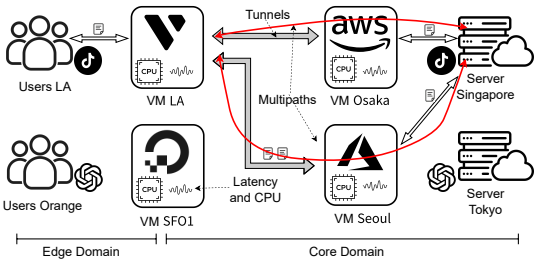


Figure 1: SkyAccel is a cross-cloud traffic acceleration platform for geographically distributed latency-sensitive workloads.

Industrial acceleration strategies generally fall into two main categories. The first deploys extensive worldwide Points of Presence (PoPs) to forward user traffic to nearby access nodes and reduce last-mile latency [10]. However, large-scale PoPs construction and maintenance require massive capital and operational investment, making this strategy impractical for small and medium-sized enterprises (SMEs). Furthermore, merely optimizing the last-mile still fails to eliminate source-to-destination transmission bottlenecks entirely. Under these conditions, SMEs commonly adopt commercial cloud acceleration services (CCASs), such as AWS Global Accelerator (AWS-GA) [8], Alibaba Cloud Global Acceleration (AC-GA) [3] and GCP Global Load Balancing (GCP-GLB) [20]. By leveraging Anycast IPs, dedicated backbone networks, health inspection mechanisms, and multi-region redundancy, these services route traffic toward healthy endpoints. As a result, they provide low latency, high availability, and stable performance at scale. Unfortunately, CCASs exhibit several practical limitations in emerging multi-cloud settings. A key limitation is excessive operational overhead significantly increases long-term costs. Another challenge is commercial acceleration capabilities are deeply bound to vendor-specific infrastructures, limiting deployment flexibility. Additionally, existing acceleration platforms are often tightly integrated with shared infrastructure services (e.g., ECS [7] or CDN [1]), which limits extensibility and customization, making it difficult to introduce service-specific control mechanisms or differentiated optimizations.

Beyond these deployment and operational limitations, existing open-source systems targeting cloud data movement (e.g., bulk data transfer) are primarily designed for throughput maximization and cost efficiency. Since throughput is an end-to-end metric determined by the entire path, achieving optimal performance requires

This paper has been accepted for publication at ACM SoCC 2026. This version includes an appendix with supplementary technical details omitted from the main text due to space constraints.

globally coordinated decision-making. As a result, these systems naturally rely on optimization-based approaches [26, 63] under relatively stable and well-provisioned infrastructure assumptions. These designs implicitly assume that network paths and node capacities exhibit limited variability, making whole-system throughput optimization both tractable and effective in practice. However, this assumption no longer holds in cost-constrained heterogeneous multi-cloud environments, where opportunistic resources (e.g., spot instances [41, 65] and low-cost alternatives [49–51, 57, 59, 73]) introduce highly dynamic and unpredictable conditions [53, 75]. This leads to a mismatch between fast-changing local dynamics and system-wide decision processes, limiting the effectiveness of unified control. First, LSWs are highly bursty and CPU-dominant, leading to rapid temporal variations in system state. Under such dynamics, control decisions increasingly lag behind rapidly evolving system states. Consequently, even small discrepancies between observed and actual conditions can lead to disproportionately suboptimal decisions [1]. Second, multi-cloud environments introduce significant heterogeneity and instability across providers, with resource conditions varying over time and space. This limits the applicability of unified control, which relies on a relatively stable and predictable system state. Third, latency-driven traffic engineering (e.g., based on K-Shortest paths [17] and maximum flow formulations [28]) tends to concentrate traffic onto a small subset of seemingly optimal paths, leading to persistent spatial load imbalance. This is a natural consequence of optimization-driven routing, where locally optimal decisions reinforce the selection of already favorable paths under uncertainty [27].

In well-provisioned data center environments, where peak CPU utilization remains considerably low (around 30%, Fig. 3), temporal fluctuations are typically absorbed locally and do not propagate, as the system operates far from saturation. Nevertheless, in cost-constrained multi-cloud settings, where both node capacity and network conditions are weakly provisioned and time-varying, model-driven mechanisms built on conventional redundancy assumptions struggle to suppress performance fluctuations, leading to localized overload and congestion propagation. This leads to a setting where performance optimization and system stability must be jointly considered under dynamic conditions. This challenge is further amplified by a multi-timescale mismatch, where globally synchronized control loops operate at relatively rigid decision intervals, while system execution reacts to rapidly evolving local dynamics. This mismatch is further reflected in the structural separation between the edge domain, which handles last-mile user access and routes incoming requests to nearby edge nodes, and the core domain, which manages backend traffic from edge nodes to service servers by performing inter-region routing and server selection. The two domains exhibit distinct characteristics in terms of latency sensitivity and resource pressure.

We observe that edge and core domains exhibit distinct control dynamics: edge control reacts to transient resource fluctuations, whereas core routing evolves according to slower end-to-end path conditions. This observation motivates an asymmetry-aware control decomposition (ACD). Rather than introducing a new hierarchical structure (e.g., proximity-based access [13] and segmented routing [44]), we build on standard edge-core infrastructures and redesign how control responsibilities are assigned. Specifically, ACD places stability control at the edge and routing optimization in the core, allowing each to operate according to its own dynamics. The edge domain operates on a rapid feedback loop with fine-grained and highly variable observations of latency and load conditions. It is therefore primarily responsible for stabilizing instantaneous fluctuations in user-facing traffic. In contrast, the core domain operates over coarser-grained and delayed aggregated signals. It is better suited for routing-level optimization that leverages long-haul path diversity to improve end-to-end efficiency and avoid persistent congestion hotspots. Crucially, these two control loops operate on different temporal scales and observation fidelities. Their naive coupling under a unified policy leads to interference between fast-reacting stability control and slow-moving routing decisions, making it difficult to jointly optimize stability and routing.

Finally, we implement SkyAccel (as shown in Fig. 1), a practical multi-cloud traffic routing system deployed across heterogeneous virtual machines (VMs) [2, 16, 21, 54]. Experimental results show that SkyAccel reduces 90th-percentile latency by up to 60.07% compared to representative commercial CCASs. In well-provisioned regions such as Asia-Pacific, SkyAccel achieves performance comparable to existing solutions, while in resource-constrained regions such as South America it yields significantly larger gains due to higher cross-cloud path diversity. It also maintains stable operation under high CPU utilization, sustaining up to 200K connections per minute on 2-core shared instances. Moreover, it reduces acceleration cost by 4–30× across different regions and service providers compared to mainstream CCASs, demonstrating improved trade-offs between latency, stability, and operational cost.

In summary, our contributions are as follows:

(1) We identify an asymmetry in heterogeneous multi-cloud acceleration systems under unstable network conditions, and propose an asymmetry-aware control decomposition (ACD) that decouples stability-critical edge control from performance-aware routing optimization in the core domain.
(2) We present SkyAccel, a provider-agnostic traffic acceleration system for LSWs that decouples routing from cloud-specific infrastructure, enabling deployment portability across cloud environments. The system is open-sourced at *https://github.com/DenebCloud/SkyAccel*.
(3) We demonstrate that SkyAccel significantly improves latency and stability compared with mainstream CCASs, while also reducing cost, making it a cost-effective alternative to hyperscale cloud acceleration services for cost-sensitive cloud users, including SMEs and resource-constrained users.

2 Background and Motivation

2.1 LSW Characteristics

This section characterizes LSWs and analyzes performance bottlenecks in existing service systems under such latency-critical

[1] Routing in complex cloud topologies is combinatorial and non-convex, so even minor runtime perturbations under high load can shift the optimal solution [6, 72].

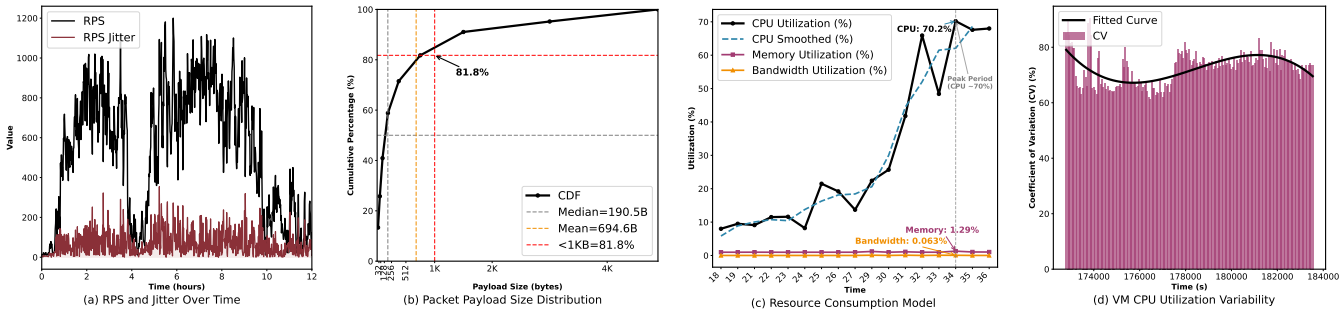


Figure 2: (a) RPS distribution of an Alibaba Cloud generative AI service with two daily peaks. (b) CDF of request packet sizes in AMTrace. (c) Resource utilization of a DO 4-core 8GB instance under short-connection stress testing with 512-byte requests. (d) Temporal variation of VM load (pre core) CV in AMTrace during peak periods, over a randomly selected subset of VMs.

conditions. We conduct an empirical study based on production traces from the Alibaba Cloud Cluster Trace Program (ACTP) [5], which spans multiple representative workload families in modern cloud environments, including recommendation inference [67], microservice-based applications [39, 40], generative AI services [69, 70], and GPU-based AI workloads [60, 61].

2.1.1 Traffic Patterns. LSWs are characterized by high request concurrency with strong temporal burstiness and predominantly small request payloads. Modern latency-sensitive services operate at massive scales, with millions of concurrent sessions in live streaming and VoD systems [35, 50, 55], while microservice-based systems often process hundreds of thousands of calls per minute [40]. Beyond scale, these workloads are highly bursty. Fig. 2(a) shows the Alibaba Cloud (AC) production generative AI trace [69], where request intensity, measured in request per second (RPS), exhibits 17.03% average adjacent fluctuation and up to 53.66% instantaneous variation, indicating strong short-term non-stationarity that is likely to cause temporary queue buildup and short bursts of congestion in the network. In addition, LSW traffic is dominated by small messages. Based on analysis of 80,079,074 samples from Alibaba Microservices Traces (AMTrace) [56], the average request payload size is 694.6 bytes, with a median of 190.5 bytes, and 81.8% of requests are below 1 KB (as illustrated in Fig. 2(b)). Similar patterns are observed in the Dagger [19, 32] microservice platform, where most RPCs are sub-KB and response packets are typically below 64 bytes. As a result, transmission time becomes negligible relative to queueing and scheduling delays, making tail latency highly sensitive to transient congestion and routing instability.

2.1.2 Resource Consumption Model. Fig. 2(c) illustrates the CPU, memory, and bandwidth overhead of a DigitalOcean (DO) instance `s-4vcpu-8gb` (4 vCPU, 8 GB memory, ~1 Gbps network) under an intentionally stressed short-connection workload with 512-byte payloads, representative of typical LSW traffic. The experiment amplifies connection-churn overhead under highly transient workloads to expose the dominant resource bottleneck more clearly. We observe that CPU utilization quickly becomes the dominant bottleneck. As CPU usage approaches saturation around 70% both memory consumption and bandwidth utilization remain near 1%. This behavior is primarily driven by per-connection processing overhead including frequent connection setup and teardown context switching kernel scheduling and interrupt handling. In contrast, the small payload size and short connection lifetime keep memory and bandwidth demands consistently low.

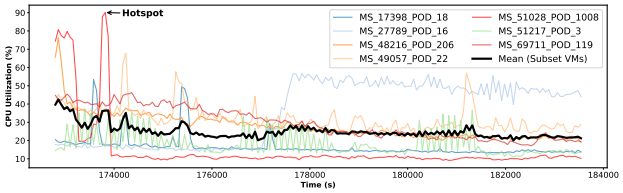


Figure 3: CPU utilization (pre core) traces of high-variability VMs during peak periods, illustrating their load dynamics.

2.1.3 Load Variability and Hotspot Behavior. We analyze load fluctuation and hotspot behaviors in the AMTrace. The cluster spans more than 460K VMs, shared across multiple services in a large-scale cloud infrastructure, which are mainly composed of 2-core (13.09%), 4-core (62.27%), and 8-core (24.64%) instances. We randomly sample 100 VMs from the cluster and focus on the evening peak period (20:00–23:00) to examine CPU utilization dynamics. As shown in Fig. 2(d), the coefficient of variation (CV) of VM load remains consistently high. Most CV values stay above 60%, and some instances exceed 80%, indicating strong temporal variability and significant load imbalance within the cluster. To further understand this variability, we examine representative Pods with the largest fluctuations. As shown in Fig. 3, these Pods exhibit highly bursty behavior, where instantaneous CPU utilization frequently spikes to 80%–90%. In contrast, the black curve, the average CPU utilization across the sampled VM subset, remains low at around 30%, highlighting the discrepancy between short-term bursts and aggregated load behavior. These results suggest that production microservice environments inherently exhibit strong temporal imbalance and burst-driven hotspots, significantly complicating scheduling and resource management. Such challenges are characteristic of modern scheduling systems and persist even in well-provisioned production deployments [70], while being further exacerbated in resource-constrained networks [49]. In light of these dynamics,

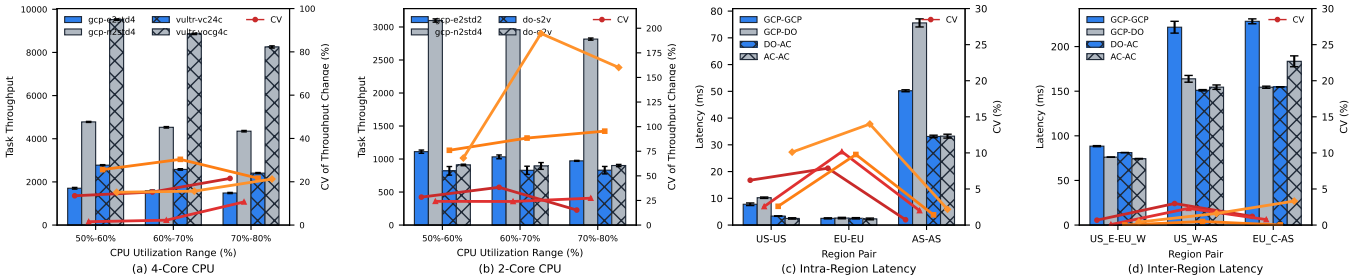


Figure 4: (a-b) Task execution throughput (sysbench CPU events/s) under varying CPU utilization. The right y-axis reports throughput CV. (c-d) Cross-region latency over intra- and inter-provider links across multiple cloud providers; the right y-axis shows latency CV.

hotspot-induced instability emerges as a key concern and must be addressed in system design.

2.2 Heterogeneity in Multi-Cloud Resources

Multi-cloud environments exhibit significant variations in compute capacity, memory availability, bandwidth, pricing, network latency, and packet loss across providers and regions [53, 75]. Prior systems [11, 26, 33, 63] in multi-cloud settings have extensively studied throughput-oriented workloads such as bulk transfers, VoD, and live streaming, focusing on metrics such as bandwidth utilization, throughput, and transmission stability. However, these systems are less suited to latency-critical interactive services, which exhibit largely different workload characteristics and performance bottlenecks. Moreover, existing research predominantly relies on high-end dedicated VMs with ample resources, such as high-end configuration instances from major cloud providers (e.g., AWS m5.8xlarge, Azure Standard_D32_v5, and GCP n2-standard-32, all provisioned with 32 vCPUs and 128 GB memory [26]), under the assumption of relatively homogeneous and over-provisioned environments [26]. However, unlike these over-provisioned settings, practical global acceleration deployments for SMEs often rely on lightweight 2-core and 4-core VMs (e.g., spot or shared-CPU instances) to achieve broad geographic coverage within a limited budget. To the best of our knowledge, such heterogeneous and resource-constrained environments remain under-explored in prior studies.

2.2.1 CPU Processing Stability. Fig. 4(a) and 4(b) analyze the processing capability and CPU sensitivity of different instance types across varying CPU utilization levels, where sensitivity is quantified by the coefficient of variation of throughput (CVTC) under standard stress-testing workloads. CVTC is a key metric for high-load traffic scheduling, as even small fluctuations in control intervals or migration volume can amplify connection surges and destabilize system performance. In terms of compute capacity, we observe significant performance differences even among instances with the same number of vCPUs. For example, the CPU task throughput of the instance Vultr `voc-g-4c` is 5.43 times higher than that of the shared-core instance GCP `e2-standard-4`. In terms of CPU sensitivity, most instance types exhibit CVTC values above 20%, indicating noticeable performance fluctuation under incremental CPU load changes. GCP instances remain relatively stable, while DigitalOcean instances show substantially higher sensitivity. To validate whether higher CPU configurations lead to reduced performance variability, we evaluate higher-core instances and find that larger-core configurations generally provide better stability. For example, both dedicated and shared 8-core Vultr instances exhibit relatively low variability, with CVTC variation remaining within approximately 10%.

2.2.2 End-to-End Network Latency Variability. In the cross-region latency measurements (Fig. 4(c) and 4(d)), we first observe a clear order-of-magnitude gap between intra-region and inter-region links, where intra-region edge domain links typically exhibit tens-of-millisecond latency, whereas inter-region core domain links incur latencies on the order of hundreds of milliseconds. Beyond this absolute latency gap, we further observe a trade-off between latency and stability, where stability is quantified by the coefficient of variation of latency (CVL). Intra-region links exhibit lower absolute latency but higher latency variability (CV up to 15%) (Fig. 4(c)). In contrast, inter-region links exhibit higher base latency but relatively lower variability (typically within 5%) (Fig. 4(d)). CVL is a critical metric for stability-aware scheduling in edge domain links, as it captures latency variability that directly impacts routing decisions. Unlike core domain backbone links, which benefit from traffic aggregation and predictable queuing behavior, edge domain links are more susceptible to transient congestion and resource contention [45]. Due to their low baseline latency, even small absolute fluctuations can result in significant relative variation, making edge links a major source of whole-level latency instability.

2.3 Asymmetry-Aware Control Decomposition

Grounding on Sec. 2.1 and Sec. 2.2, LSWs and multi-cloud environments jointly induce coupled temporal and spatial variability. Bursty, CPU-intensive workloads interact with heterogeneous compute and network conditions across providers, leading to asymmetric sensitivity in control decisions across compute and network subsystems. This coupling further suggests that a single unified scheduling policy may struggle to accurately capture system behavior across both time and space. As a result, workload fluctuations and infrastructure heterogeneity jointly amplify control uncertainty, making robust scheduling and whole-system performance guarantees challenging under LSWs.

2.3.1 Edge-Core Control Decomposition. The system naturally exhibits structural separation along two complementary dimensions.

First, network latency varies across path segments, where different domains show distinct latency magnitudes and sensitivity to temporal fluctuations (Fig. 4(c) and 4(d), where intra-region paths exhibit greater latency variation than inter-region paths). Second, resource and interaction patterns differ across system components, with edge segments tightly coupled to user-facing, compute-intensive operations, while core segments primarily handle inter-node routing and traffic scheduling (see Fig. 2(b)). These observations suggest an opportunity for separating control responsibilities across edge and core domains. Building on this structure, we introduce a layered control formulation that organizes system scheduling into two loosely coupled components: stability-oriented local control at the edge and latency- and path-diversity-aware routing optimization in the core. This decomposition transforms end-to-end path control into two subproblems operating at different temporal and spatial granularities. Moreover, this design is also justified under the practical assumption that edge domain user ingress incurs higher computational overhead than intra-cloud data forwarding based on tunnel encapsulation and traffic aggregation in the core domain. We observe that the edge domain incurs approximately 3× higher CPU overhead than the core domain, attributable to the packet aggregation mechanism in the core-domain data plane, as empirically validated in Sec. 5.2. Within this formulation, the edge domain focuses on stabilizing stochastic workload dynamics and is implemented using stochastic network optimization (SNO) techniques [43]. In contrast, the core domain addresses routing under path contention and is modeled as a conflict-aware maximum-flow problem (MFPC) [12].

2.3.2 Necessity of SNO and MFPC. The edge and core domains exhibit distinct control requirements, motivating different optimization frameworks. The edge domain operates under stochastic workloads with delayed observations, making prediction-driven scheduling impractical [18, 38], and is formulated as an online stability control problem using Lyapunov-based drift control for queue stability. Conversely, the core domain is a spatial resource allocation problem over shared network infrastructure, where competing flows induce path-level contention that cannot be captured by classical shortest-path or max-flow formulations, driving a MFPC formulation. Together, SNO and MFPC address complementary temporal and spatial aspects of system-wide control.

3 System Design

3.1 Design Overview

SkyAccel is an application-layer overlay traffic acceleration system designed for latency-sensitive services in heterogeneous multi-cloud environments. It leverages geographically distributed relay nodes on cloud VMs across multiple providers and regions to dynamically orchestrate user traffic, without modifying the underlying Internet routing infrastructure or cloud network stack. Fig. 5 shows that the overall system design of SkyAccel, which consists of three functional planes: the information plane, the control plane, and the data plane. The information plane maintains network and resource states required for scheduling decisions, while the control plane is further decomposed into edge and core domains, separating fast local decisions from slower global optimization. The data plane executes actual traffic forwarding through overlay tunneling and relay transmission mechanisms.

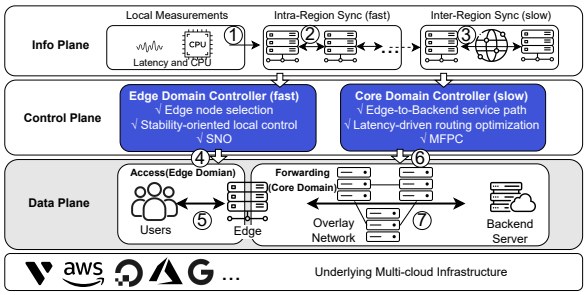


Figure 5: SkyAccel's three-plane system architecture, showing the hierarchical control structure and workflow of traffic scheduling.

3.1.1 Information Plane Design. The information plane maintains a unified state abstraction consisting of latency, load, and reachability metrics, which are periodically refreshed with bounded staleness. It first collects local node state, including system-level latency measurements and real-time CPU utilization, from local nodes (①). Next, it performs intra-zone state synchronization across nodes within the same geographic region to enable fast local control loops (②). Finally, inter-zone synchronization ensures global state visibility across all cloud regions (③). The edge and core domains synchronize at second-level and 5-second granularities, respectively. If a node fails to synchronize its state in a given round, it is evicted from the active topology to prevent stale state from affecting subsequent decisions. This hierarchical design aligns with the decomposed control structure, allowing time-sensitive edge domain decisions to rely on local state while deferring global consistency to the core domain stage.

3.1.2 Control Plane Design. To instantiate the two-stage ACD architecture, the control plane is logically decomposed into an edge domain controller and a core domain controller. The edge domain controller performs per-request, low-latency ingress selection based on local network conditions, enabling fast determination of entry points (④). In contrast, the core domain controller operates on a slower, aggregated control timescale, generating relay execution plans from system-wide state to coordinate multi-hop forwarding toward backend services (⑥).

3.1.3 Data Plane Design. The data plane of SkyAccel consists of two components: the access layer (⑤) and the forwarding layer (⑦), corresponding to the edge and core domains, respectively. The access layer handles user ingress and nearby edge-node access, while the forwarding layer handles intermediate-hop relay across overlay nodes and final-hop delivery to backend servers. Traffic forwarding is implemented through overlay relay tunnels between distributed nodes, together with packet aggregation mechanisms to improve cross-zone transmission efficiency.

3.1.4 Traffic Pipeline. We describe the end-to-end request flow, which integrates the information, control, and data planes as shown in ①–⑦. (i) A user is first directed to a nearby edge node via

GeoDNS-based [14] nearest-region resolution . (ii) The nearest edge domain controller returns an access routing decision, and the user connects to the selected edge node. (iii) The edge node then queries its core domain controller to obtain a forwarding route. (iv) The selected route is used to forward traffic via overlay tunnels before reaching the backend service server. (v) The return path follows the reverse direction of the forward routing path.

3.2 Edge Domain Scheduling (EDS)

We formulate EDS as an online scheduling problem that balances congestion stability and user-to-edge latency.

3.2.1 Virtual Queue Formulation. We introduce a virtual queue to represent congestion pressure under coupled request allocation and runtime load dynamics. Specifically, a group of N edge nodes is represented by $Q(t) = (Q_1(t), \ldots, Q_N(t))$, where $Q_i(t)$ represents the accumulated congestion pressure of node i relative to a target load level θ. The virtual queue evolves as $Q_i(t+1) = \max(Q_i(t) + x_i(t) - \theta, 0)$, where $x_i(t)$ denotes the normalized CPU load added to the node i by the requests assigned to it at time t, and θ is the target load level.

3.2.2 Per-Slot Scheduling Policy. The idealized congestion-stability objective can be characterized by the evolution of $Q(t)$. To analytically characterize the trade-off between congestion pressure and latency under stochastic arrivals, we use the Lyapunov drift-plus-penalty framework [43], which motivates the following idealized per-slot objective:

$$\min_{\Delta req_i(t)} \sum_{i=1}^{N} \left(w_i Q_i(t) x_i(t) + V \cdot delay_i(t) \right), \tag{1}$$

where $w_i Q_i(t) x_i(t)$ represents the congestion cost associated with assigning load to node i, and $delay_i(t)$ represents the corresponding user-to-node latency. Here, w_i is a node-specific weighting factor, and V controls the relative importance of latency.

3.2.3 Online Approximation. Solving Eq. 1 exactly at runtime is computationally expensive because request allocation and the resulting CPU load are tightly coupled. This coupling makes the per-slot optimization nonlinear and combinatorial and requires continuously tracking the global virtual-queue state, which is costly for large-scale online deployment. Theoretically, the virtual-queue formulation provides an analytical abstraction, while practical implementations can use directly measured CPU-utilization states [47]. Specifically, let $C_i(t)$ denote the measured CPU utilization of node i, and let $\Delta C_i(t)$ denote its short-term change.

To derive an implementable heuristic from Eq. 1, we preserve its two cost components and approximate each using directly measurable runtime states. The congestion term $Q_i(t)x_i(t)$ is heuristically represented by a nonlinear CPU-utilization penalty that increases with the current utilization and its short-term variation, while the latency ($delay_i(t)$) term is represented by a nonlinear penalty on the measured latency relative to the target latency. This yields the following scheduling score:

$$S_i(t) = w_i \exp\left(\alpha_i \left(\frac{C_i + |\Delta C_i|}{M_c} - 1\right)\right) + V \exp\left(\beta \left(\frac{lat_i}{lat^g} - 1\right)\right), \tag{2}$$

where M_c is a core-dependent reference utilization level, α_i and β are sensitivity parameters, $lat_i(t)$ is the measured user-to-node latency, and lat^g is the target latency. Incoming requests are assigned according to a softmax over the negative scheduling scores, where temperature T controls the trade-off between exploration and score sensitivity. The exponential form increasingly penalizes high-utilization and high-latency states, making the scheduler more sensitive to overloaded nodes and latency degradation. Thus, Eq. 2 heuristically maps the congestion and latency components of Eq. 1 to measurable CPU-utilization and latency states, respectively. This design avoids explicit per-slot optimization and provides a lightweight mechanism for routing requests according to measured congestion and latency under dynamic workloads.

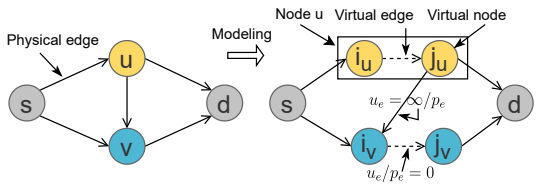


Figure 6: Network model. Virtual edges impose finite capacity (u_e) for node-level throughput, while original edges carry infinite capacity with per-unit transmission cost (p_e).

3.3 Core Domain Scheduling (CDS)

Traffic is routed over competing backbone paths under feasibility constraints. The objective is tradeoff-aware routing leveraging path diversity to reduce latency and mitigate congestion over longer timescales, all within a unified architecture.

3.3.1 Modeling as a MFPC. We transform the constrained multi-path problem (CMPP) into a MFPC. As illustrated in Fig. 6, we model the network as a directed graph $G = (V, E)$ with ingress node s and destination set D. To capture node-level capacity constraints, each intermediate node $v \in V_m = V \setminus (s \cup D)$ is split into an input-output pair (i_v, j_v) connected by a virtual edge, yielding an extended graph $G' = (V', E')$. In G', virtual edges model node capacity limits, while original edges represent transmission links with latency costs.

$$\begin{cases} \max \quad \mathcal{F} & \\ \text{s.t.} \quad \sum_{(i,j)\in E'} f_{ij} = \sum_{(j,i)\in E'} f_{ji}, & \forall i \in V_m \\ \quad f_{ij} \le \theta_a(i,j) u_{ij}, & \forall (i,j) \in E' \\ \quad L(P_{s\to d}) \le \theta_L(d), & \forall P_{s\to d} \in \mathcal{P}_s,\ d \in D \\ \quad \sum_{j:(j,d)\in E'} f_{jd} \le K_d, & \forall d \in D \end{cases} \tag{3}$$

3.3.2 MFPC Formulation. We formulate the problem as Eq. 3, where f_{ij} denotes the flow allocated on edge (i, j), and u_{ij} denotes its capacity. Flow conservation enforces balance at every intermediate node $i \in V_m$. Edge utilization is constrained by an edge-specific threshold $\theta_a(i, j) \in [0, 1]$, preventing excessive traffic concentration and mitigating hotspot formation. The latency constraint ensures that each selected path $P_{s\to d} \in \mathcal{P}_s$ satisfies the destination-specific latency requirement $\theta_L(d)$. The destination capacity constraint bounds the total incoming traffic at each destination by K_d, capturing resource limitations such as processing capacity or access bandwidth. The

objective $\mathcal{F}$ captures the overall utility of a candidate solution under these constraints. We formulate $\mathcal{F}$ as a weighted composite objective: $\mathcal{F} = w_p N_{\text{path}} + w_d D_{\text{diversity}} - w_c P_{\text{congestion}} - w_l P_{\text{latency}}$, where N_{path} is the number of admitted paths, $D_{\text{diversity}}$ measures path diversity, and $P_{\text{congestion}}$ and P_{latency} penalize congestion and excessive latency, respectively. The weights w_p, w_d, w_c, and w_l determine the relative importance of the four objectives. Thus, the heuristic balances traffic admission, path diversity, congestion, and latency[2]. The utilization-aware capacity constraints limit the load carried by individual links, encouraging the heuristic to distribute admitted paths across multiple feasible routes and thereby reducing the risk of backbone hotspots.

3.3.3 Online Execution of MFPC. We enforce MFPC constraints through a lightweight online mechanism driven by local congestion signals. Each node v maintains a dynamic participation factor $\theta_a(v)$ that regulates the number of admissible paths traversing the node based on runtime telemetry such as queue length and link utilization, enabling adaptive mitigation of hotspot formation. For online execution, we approximate the continuous flow constraints using a discrete path-counting model, where each admitted path consumes one unit of capacity and the number of paths traversing edge (i, j) is bounded by $\theta_a(i, j)u_{ij}$. This avoids the need for traffic prediction or flow-based optimization. A candidate path is admitted only if adding it preserves the capacity constraints on all traversed edges, the path-latency constraint, and the destination-capacity constraint; otherwise, it is rejected. Overall, the mechanism provides an online decentralized MFPC approximation over finite candidate paths, maintaining feasibility during incremental selection.

3.3.4 Complexity and Approximation. The MFPC problem with conflict-style coupled capacity constraints is strongly NP-hard. We therefore follow the low-complexity incremental path-selection framework of carousel greedy [12]. Specifically, starting from an empty solution, the heuristic iteratively evaluates the remaining candidate paths, selects the path with the best objective value among those satisfying the current capacity and conflict constraints, adds it to the solution, and updates the residual capacities and affected constraints. The process terminates when no feasible candidate path remains. The detailed algorithmic implementation is provided in Appendix A. Let P denote the number of candidate paths, K the number of paths in the final solution, and L the maximum hop length of any path. Each iteration requires $O(PL)$ time to evaluate candidate paths and adds one path to the solution, resulting in an overall time complexity of $O(KPL)$.

4 Implementation

(1) System Architecture. The three logical planes described in Sec. 3.1 are implemented through three deployment components. The information plane is not deployed as a standalone service; instead, its functionality is realized through distributed telemetry collection and state synchronization mechanisms, which are distributed across the different deployed components. SkyAccel adopts a hierarchical deployment architecture consisting of three components: Controller, Collector, and Proxy. Each logical node is composed of multiple co-located VMs. A single Controller is deployed per node to maintain system state and perform scheduling decisions. Each VM hosts both Collector and Proxy modules. The Collector acts as a lightweight telemetry agent that collects latency and resource measurements and reports them to the Controller. The Proxy handles edge access, overlay relay, and traffic forwarding.

(2) CPU and Latency Measurement. SkyAccel collects CPU and latency measurements through the Collector. Network measurements are decomposed into edge domain and core domain components. The core domain latency is obtained via active probing, while the edge domain latency is inferred from passive observations during user connections (e.g., connection setup latency). These measurements are aggregated to estimate overall user-to-edge latency. In addition, the system collects per-second CPU utilization along with fluctuation statistics to characterize node-level load dynamics. Together, these network and compute signals serve as a proxy for system congestion and stability, which are used by the Controller for scheduling and routing decisions.

(3) Transport Layer Design. The Proxy adopts QUIC [31] as the underlying transport for tunnel-based forwarding, enabling efficient stream multiplexing and improved robustness under heterogeneous cross-region network conditions. Compared with TCP-based tunneling [48], QUIC provides user-space stream multiplexing and mitigates head-of-line blocking at the transport layer, making it well-suited for cloud deployment scenarios. In addition, the Proxy operates at the transport layer and supports both TCP and UDP traffic, performing connection-level forwarding (i.e., without parsing application-layer semantics). This allows it to transparently carry modern services such as HTTP, WebRTC [58], and gRPC [23].

(4) Request Aggregation and Packet Header. At the packet layer, the Proxy performs request aggregation by merging multiple application level packets into a single transmission unit to reduce per-packet processing overhead and improve transmission efficiency. The merged packet header is composed of fixed and variable segments. The fixed segment carries forwarding metadata, including `HopPos` and `HopList` for execution ordering, as well as `Port` and `Protocol` for destination identification. The variable segment carries per-request metadata, where `RequestID` identifies individual requests within the aggregated packet.

5 Evaluation

SkyAccel is evaluated in heterogeneous multi-cloud environments to validate its acceleration performance, efficiency, and robustness under realistic deployment conditions. Our evaluation focuses on four aspects: (i) end-to-end latency improvement over representative CCASs (Sec. 5.1); (ii) data plane performance and efficiency under increasing traffic intensity (Sec. 5.2); (iii) layer-specific control behavior, including stability and load balancing in the edge domain, and latency optimization and routing diversity in the core domain (Sec. 5.3 and Sec. 5.4); and (iv) ablation studies of the ACD architecture (Sec. 5.5), showing its impact on stability and core-domain route performance. We deploy a tiered heterogeneous multi-cloud testbed across GCP, Alibaba Cloud, Vultr, and DigitalOcean. Each node runs on a dedicated 2- or 4-core Debian 12 VM public WAN links with CUBIC congestion control, using 512-byte payloads.

[2]While Fig. 6 shows per-unit transmission prices p_e at the edge, the optimization does not explicitly include cost terms. We discuss this design choice in Sec. 6.

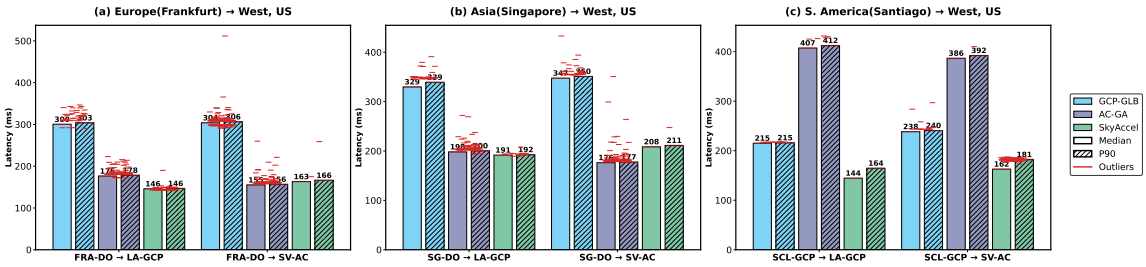


Figure 7: End-to-end latency comparison of GCP-GLB, AC-GA, and SkyAccel under geographically distributed client access.

5.1 End-to-End Latency Improvement

5.1.1 Experimental Setup. Existing cloud-based acceleration services tightly couple control logic and forwarding within provider-specific execution stacks, enabling performance-aware routing within a single cloud domain but offering best-effort handling of cross-cloud traffic. We compare SkyAccel against two representative CCASs, GCP-GLB and AC-GA, representing different global traffic delivery designs, where AC-GA focuses on latency optimization and GCP-GLB emphasizes global traffic steering and load balancing (we omit direct client-to-backend paths, as multi-segment forwarding [44] with nearby ingress [13] has already been shown to outperform direct cross-region access). The testbed spans multiple geographic regions. Backend services are deployed on Alibaba Cloud (Silicon Valley, SV) and GCP (Los Angeles, LA). Client workloads originate from DigitalOcean instances in Frankfurt and Singapore, as well as GCP in Santiago. SkyAccel's control and data planes are deployed on Vultr to coordinate cross-cloud routing and traffic forwarding. The evaluation covers both peak and off-peak periods, with acceleration performance measured every 30 seconds.

5.1.2 Performance Analysis. Figure 7 shows that SkyAccel exhibits stable latency across regions, with median and 90th-percentile latencies differing by about 10% in low-quality regions such as South America, and negligible differences in well-provisioned regions. This result is consistent with SkyAccel's design choice of operating over the public Internet rather than dedicated network paths. SkyAccel further achieves consistently low latency across geographically distributed access scenarios, with the most significant gains in low-quality network regions. In South America, it achieves a 90th-percentile latency of 0.164–0.181s for South America-to-West US traffic, reducing latency by 23.49–60.07% compared with GCP-GLB and AC-GA. In well-provisioned regions (Europe and Asia-Pacific), it performs comparably to AC-GA while still outperforming GCP-GLB by 40–50%. SkyAccel also provides substantial cost benefits, maintaining a stable cost of approximately $0.02 per GB across all regions over Vultr. In contrast, GCP-GLB and AC-GA are 4–9× and 10–30× more expensive, respectively [4, 22][3]. For example, GCP-GLB incurs around $0.181 per GB for South America-to-North America traffic, primarily due to inter-region transfer ($0.14/GB) and load balancing ($0.016/GB) overheads. SkyAccel instead leverages flexible multi-cloud routing instead of premium cloud networking, reducing cost to the baseline inter-region forwarding cost. Overall, SkyAccel achieves an attractive latency-cost trade-off across all evaluated regions, enabled by its flexible cross-cloud routing and resource strategy.

[3]This comparison uses per-GB transfer pricing and does not reflect total system cost. It excludes SkyAccel's overheads and ignores CCAS managed services such as SLA, DDoS protection and vendor support. These costs are expected to be relatively small compared with bandwidth-transfer costs, but are difficult to quantify consistently at the current prototype stage.

5.2 Data Plane Efficiency Analysis

5.2.1 Setup and Workload. The experiments are conducted on shared VM instances with 2-core (Vultr `vc2-2c`) and 4-core (GCP `e2-standard-4`) configurations. SkyAccel is evaluated under cross-region traffic between cloud regions, where all requests are routed through SkyAccel for end-to-end forwarding. To generate connection-intensive workloads, clients repeatedly establish short-lived TCP connections and close the connections immediately after receiving server responses (such short-lived connections are common in request-response services and impose substantial connection processing and CPU overhead on the system, making them effective for stressing the system under high request rates). We vary the workload intensity from 50K to 200K TCP connections per minute (CPM). Each workload configuration runs for 5 minutes, during which we measure CPU utilization on access and forwarding nodes and randomly sample 2% of connections to measure request-response latency (RRL) observed by clients.

5.2.2 Aggregation Window Effects on CPU-Latency Trade-off. Increasing the aggregation window size effectively reduces overall system CPU utilization at the cost of higher mean and tail latency. As shown in Fig. 8(a)–(d), under a 200K CPM workload on the 2-core VM, enlarging the aggregation window from 0 ms to 100 ms decreases access layer CPU utilization from approximately 131.2% to 87.5%, while the tail latency increases from 134 ms to over 364.8 ms. Packet aggregation amortizes per-request processing overhead at the cost of additional queuing delay, which becomes more pronounced under high workloads. These results demonstrate that the aggregation window serves as an effective tunable parameter for balancing CPU efficiency against latency, enabling SkyAccel to adapt flexibly to diverse operational requirements.

5.2.3 Layered CPU Overhead Distribution. CPU utilization is heavily dominated by the access layer; the forwarding layer remains low-overhead for all configurations. Moreover, packet aggregation further increases the access layer's proportion of the overall system CPU usage. For instance, as illustrated in Fig. 8(g)–(h), under a 200K

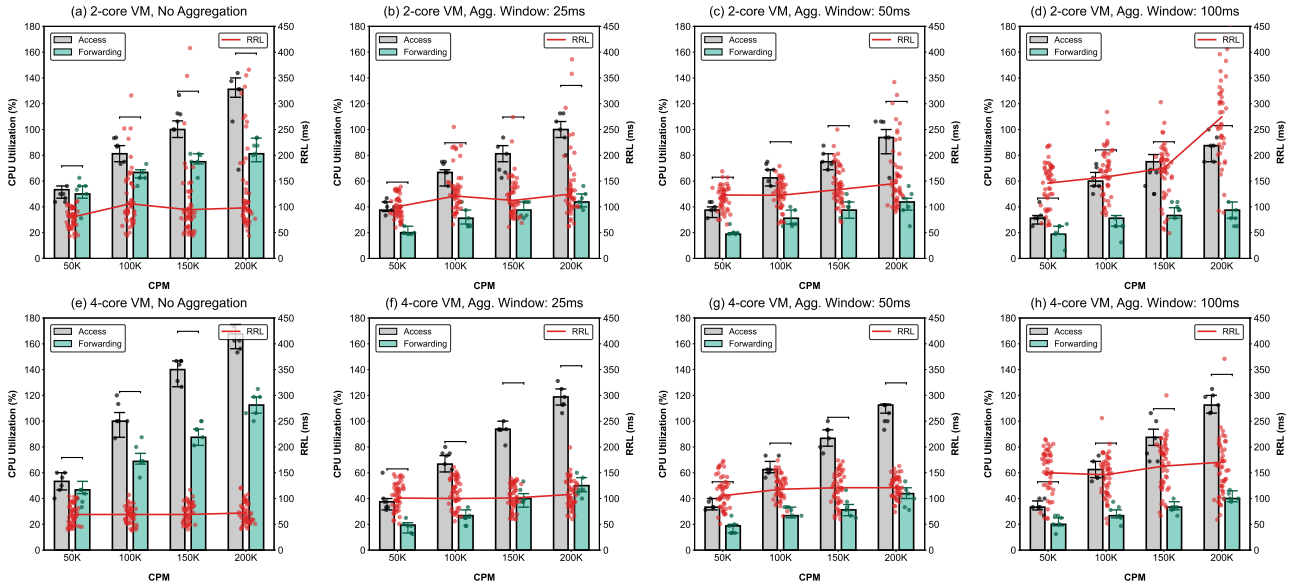


Figure 8: SkyAccel `Proxy` performance under different aggregation windows (0 ms, 25 ms, 50 ms, and 100 ms) under load testing, showing how aggregation affects packet merging behavior, CPU utilization across the access and forwarding layers, and request-response latency.

CPM workload on the 4-core VM, the access-to-forwarding CPU ratio increases from 2.57× to 2.81× as the aggregation window grows from 50 ms to 100 ms. This is primarily because multiple connection flows are first aggregated and then forwarded through tunnels, which substantially reduces CPU consumption in the forwarding layer. As a result, the forwarding layer consistently accounts for less than 30% of the total system CPU load. Due to this low utilization, it requires minimal resources for stability maintenance tasks and retains headroom for path-level optimizations in core domain.

5.2.4 Multi-core Efficiency and Latency Stability. Increasing CPU resources significantly improves both throughput and latency stability. At a workload of 200K CPM with a 100 ms aggregation window, the access layer sustains a higher aggregate CPU load, increasing from 87.5% on the 2-core VM to 112.5% on the 4-core VM. Meanwhile, the median RRL decreases from 274.5 ms (first quartile, Q1 = 194.2 ms; third quartile, Q3 = 364.8 ms) to 170.5 ms (first quartile, Q1 = 133.8 ms; third quartile, Q3 = 203.2 ms), with a substantial reduction in tail latency (see Fig. 8(d) and (h)). This enhanced stability stems from improved parallel processing capability and more effective packet aggregation at the access layer, which effectively mitigates CPU contention under bursty traffic. The results demonstrate that higher-spec CPUs can substantially stabilize latency. Furthermore, these findings highlight the challenges of simultaneously achieving high performance and stable latency when operating under constrained CPU resources in low-spec VM configurations.

5.3 Edge Scheduling under Dynamics

5.3.1 Setup and Control Parameters. The load testing client is deployed in Hong Kong. Edge nodes are distributed across multiple Asian regions, including Seoul (Vultr `vc2-2c`), Tokyo (GCP `e2-standard-2`), and Singapore (DigitalOcean `s-2vcpu`), each configured with 2 vCPUs to emulate resource-constrained edge environments. The backend server is hosted in Oregon, US. Given the intercontinental RTT between Asia-Pacific and the US West Coast (approximately 200 ms in Fig. 4(d)), we set the aggregation window to 50 ms to balance responsiveness and batching efficiency under WAN latency constraints. The control parameters in Eq. 2 are set as follows: $M_{2c} = 60\%$, $M_{4c} = 80\%$, $\alpha_{2c} = 2.0$, $\alpha_{4c} = 3.0$, $lat^g = 20$ ms, $\beta = 0.2$, and $(w_i, V, T) = (1, 1, 2)$. This configuration is calibrated using evaluation results in Fig. 4(c) and Fig. 8, where intra-region latency ranges from 20 to 50 ms, and 2-core and 4-core nodes achieve optimal performance at 60% and 80% CPU utilization, respectively. We map these empirically observed operating regions into conservative control thresholds to avoid instability-prone regimes. Under this setting, the scheduling score is approximately within $[0, 2]$, enabling a unified scale for softmax-based routing.

5.3.2 Workloads and Baselines. To evaluate robustness under dynamic workload perturbations, we extract three representative anomaly traffic patterns from high-variance phases of AMTrace CPU traces: high-frequency oscillatory traffic (HFO), burst-surge traffic (BST), and sustained-surge traffic (SST), as shown in Fig. 3. These phases are identified using a sliding-window variance analysis over CPU utilization traces, selecting segments exceeding a predefined threshold. The resulting patterns capture oscillatory, bursty, and sustained high-load regimes in production traces. We categorize edge routing strategies into two classes: memoryless stochastic selection and history-aware smoothed control. We adopt two baselines: (i) the Power-of-Two Choices (P2C) policy [42], which randomly samples two candidate nodes and selects the one with lower instantaneous load; and (ii) an EWMA-based policy [25], which applies exponentially weighted moving averages of CPU utilization

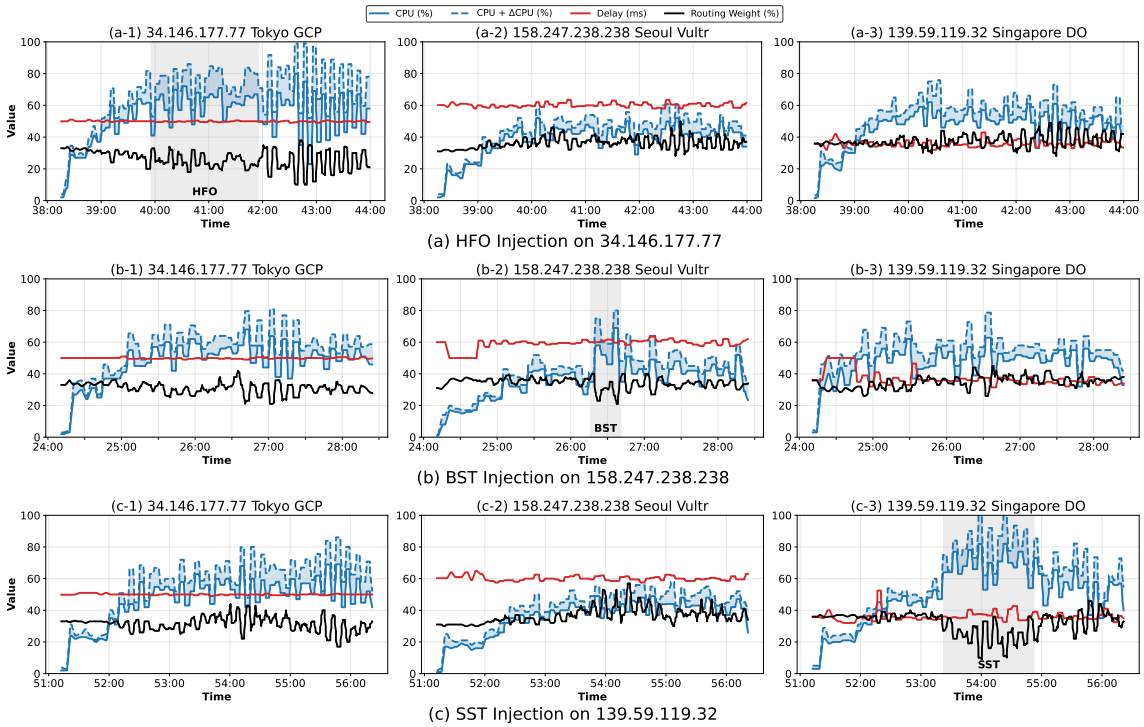


Figure 9: System dynamics of CPU, latency, and routing weights under different anomalous traffic patterns.

and latency for routing decisions. We evaluate EDS against these baselines under workload fluctuations, focusing on traffic redistribution, CPU load balance, and user-to-edge latency. Each workload scenario is evaluated over a 5-minute period, with telemetry collection and routing decisions performed every 1 second.

5.3.3 Traffic Redistribution and Runtime Stability. As shown in Fig. 9, traffic allocation remains stable under increasing load up to the critical threshold $M_{2c} = 60\%$, converging to a balanced operating point (solid black curve). At the critical threshold, the system transitions into a dynamic equilibrium, where it adaptively converges toward an efficient load-balanced allocation. Meanwhile, geographically distributed nodes exhibit a CPU-latency trade-off, where Singapore DigitalOcean achieves lower latency but higher CPU usage, while Seoul Vultr provides higher computational capacity but higher latency. Traffic converges to a balanced distribution, with Seoul and Singapore each carrying 40% of the total load, while Tokyo carries the remaining 20%. Fig. 9(a-c) illustrates system dynamics under different perturbation injections. After perturbations are introduced, traffic is redistributed across remaining nodes, and the system gradually recovers to a stable operating state once the perturbations subside. Among all cases, the Tokyo node under abnormal traffic injection exhibits relatively larger fluctuations, while the overall system response forms a narrowing envelope (see Fig. 9(a-1), (b-1), (c-1)) indicating strong robustness against external disturbances. Minor oscillations during recovery are caused by transient load rebalancing as traffic shifts back to the recovered node, further demonstrating the system's dynamic stability and elastic control behavior. This observed stability can be attributed to the underlying control design of EDS. The introduction of ΔCPU effectively captures short-term temporal variations in system load, transforming the control problem from a static state-based formulation into a dynamic feedback system with inherent damping. Together with the bounded exponential scoring function and per-slot online updates, EDS performs stable tracking of a time-varying equilibrium without requiring exact global optimization.

5.3.4 Comparison with Baselines. Fig. 10 and Table 1 jointly evaluate system behavior under HFO perturbations, which represent a typical workload fluctuation scenario, from three perspectives including tail latency, control stability, and resource imbalance. Overall, EDS achieves the best performance across all metrics. It maintains lower tail latency while reducing policy oscillations, indicating more stable routing under dynamic workloads. EWMA exhibits instability, with latency spikes and higher control variance due to overreaction to transient load changes. Although P2C achieves comparable average latency, it suffers from policy oscillation and resource imbalance, leading to hotspot accumulation. From a system perspective, these results highlight a key trade-off in baseline designs. Methods that react aggressively to instantaneous load (EWMA) tend to amplify instability, while stateless or weakly adaptive policies (P2C) fail to suppress long-term imbalance. In comparison, EDS achieves a more balanced trade-off between responsiveness and stability, resulting in smoother traffic redistribution and improved system robustness under perturbations.

5.4 Routing Behavior in Core Domains

5.4.1 Experimental Setup. We evaluate CDS on two representative SNDlib topologies [24], `native` and `cost266`, which consist of 12 and 39 nodes, respectively, representing United States and European

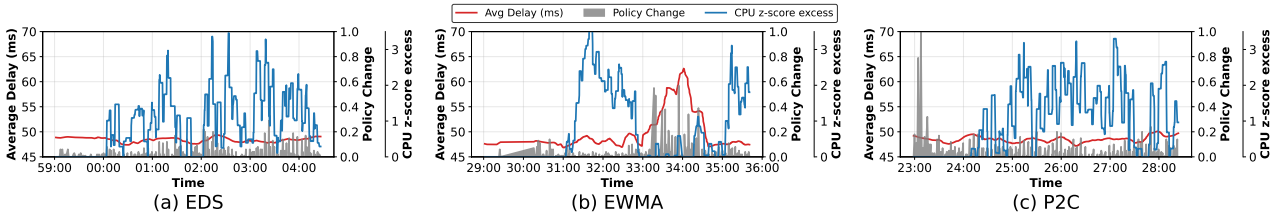


Figure 10: Runtime stability under perturbations. Delay, routing changes, and CPU hotspot severity for EDS, EWMA, and P2C.

Table 1: Performance and stability comparison under perturbations (results correspond to Fig. 10.)

Metric (↓)	EDS	EWMA	P2C	EWMA Gap	P2C Gap
Latency Tail (P99, ms)	**50.39**	78.49	51.34	+55.8%	+1.9%
Policy Oscillation (P99)	**0.24**	0.55	0.79	+131.9%	+232.2%
CPU Imbalance (Std)	**0.80** (**1.03** [**0.97**, **1.09**])	1.08 (1.18 [1.10, 1.25])	0.94 (1.10 [1.05, 1.16])	+36.5%	+17.3%
Hotspot Severity	**750.85**	943.04	1087.41	+25.6%	+44.8%

Metric definitions: Latency Tail (P99) measures tail latency of user requests (randomly sampled at a 2% sampling rate); Policy Oscillation (P99) captures tail instability of routing decisions; CPU Imbalance (Std) measures short-term variability of positive CPU z-scores, with sample mean and 95% confidence interval shown in parentheses; Hotspot Severity quantifies cumulative resource overloading.

network infrastructures[4] We compare CDS against two representative routing paradigms inspired by LiveNet [33] and ONEWAN [29]. The LiveNet-style baseline performs per-destination routing by selecting among K-shortest paths using local latency- and load-aware ranking. In contrast, the ONEWAN-style baseline follows a solver-chaining approach, first constructing a candidate path set via a maximum-flow formulation, and then performing diversity-aware selection over the resulting candidates under a network-wide load-aware objective. These baselines represent two contrasting design philosophies, namely locally adaptive heuristic routing and globally coordinated optimization-based routing, providing a comprehensive comparison against CDS. We adopt a single-source-to-multi-destination routing setting, which is common in distributed computing workloads. In each trial, one source is randomly selected, and 50% of the remaining nodes are uniformly sampled as destinations. Experiments are conducted with every node serving as the source, resulting in one trial per source node for each topology. All methods are evaluated under identical topology and workload configurations for fair comparison. For the `native` and `cost266` topologies, $\theta_L(d)$ is set to 1.5× the average latency of the top-3 KSP paths from s to d, $K_d = 2$, and $\theta_a(i, j)$ is set to 1 below 60% CPU utilization ($M_{2c} = 60\%$, $M_{4c} = 80\%$) and reduced linearly for highly utilized nodes. The objective weights are $(w_p, w_d, w_c, w_l) = (10, 1, 1, 0.1)$. We impose no hop constraint, allowing paths to reach most destinations in these edge-sparse topologies.

5.4.2 Latency-Diversity Trade-off. We examine how CDS navigates the trade-off between path latency and diversity under shared infrastructure constraints. As shown in Fig. 11, different routing designs exhibit distinct operating regions in the latency-diversity space. LiveNet-style routing achieves lower RTT but limited path diversity, whereas ONEWAN-style routing improves diversity at the cost of higher latency. CDS consistently operates in a more balanced region, achieving a better overall trade-off. We observe a small number of outliers in these subgraphs, expected given the inherent stochastic nature of hybrid greedy and kernel-based search methods. Importantly, they do not affect the overall Pareto trend, which CDS continues to dominate. This behavior is attributed to CDS's global path selection mechanism, which jointly optimizes latency and diversity under a unified objective. Unlike layered designs that decouple routing optimization from path diversity control, CDS's coupled design enables explicit coordination of both dimensions. At the system level, higher path diversity improves robustness and reduces hotspot formation under shared infrastructure constraints, allowing CDS to maintain a stable operating region while supporting tunable trade-offs between latency and diversity.

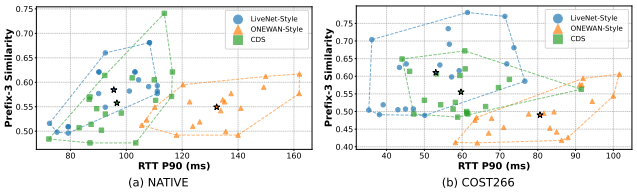


Figure 11: RTT vs. path diversity across routing strategies. The y-axis plots Prefix-3 Similarity, the path similarity of the first three hops where major path shifts at edge and cross-domain gateways.

5.5 Effectiveness of ACD

5.5.1 Ablation Study Design. To isolate the effects of control coupling and update frequency, we consider three configurations. The experiment uses workloads with US West clients accessing an HK origin via the LA access point and relay nodes in Seoul and Singapore. We inject bursty load at the Seoul edge to evaluate the

[4] `native` uses real deployments; cities without GCP/Vultr regions are approximated with the nearest region (DNVRng → GCP west3, KSCYng → central1). For `cost266`, link latencies are derived from geographic distances via SNDlib node coordinates.

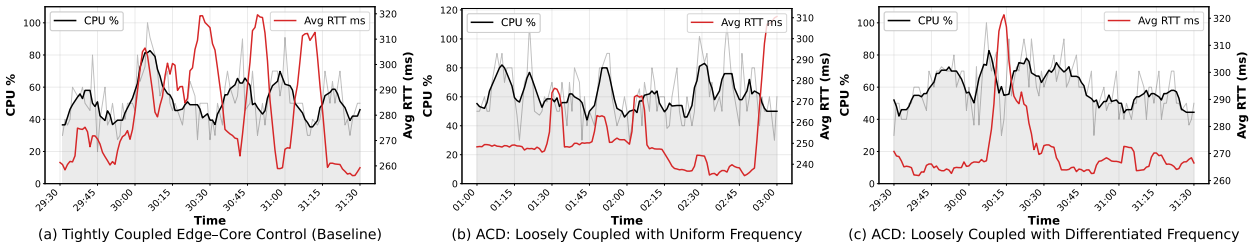


Figure 12: ACD effectiveness under bursty workloads. The left y-axis denotes: Seoul relay CPU; the right y-axis denotes: LA–HK RTT.

system's response to resource pressure. (i) A tightly coupled baseline, where the edge and core domains use CPU and latency signals with 5-s updates. (ii) A decoupled design, where the edge uses CPU-aware control and the core uses edge-to-end latency, with both loops operating at 5-s. (iii) The same decoupled design with a 1-s edge loop and a 5-s core loop, allowing us to isolate the effects of control decoupling and differentiated update frequencies.

5.5.2 Ablation Analysis. Fig.12 compares Seoul relay CPU utilization and LA–HK RTT across the three configurations. In Fig.12(a), with a 5-s control period, tight coupling causes pronounced oscillations in both CPU and RTT, as transient CPU fluctuations repeatedly trigger routing adjustments and further redistribute the workload. Fig.12(b) removes the cross-domain coupling while keeping the 5-s control period unchanged. The edge loop still responds slowly to bursty load, so CPU and RTT continue to fluctuate, but with substantially lower amplitudes than in Fig.12(a), as the decoupled control prevents CPU fluctuations from directly triggering core-level routing adjustments. Fig.12(c) further reduces the edge control period from 5-s to 1-s while retaining the decoupled design, allowing CPU pressure at the Seoul relay to be mitigated more promptly by redistributing the load across the Northeast Asia access group; both CPU and RTT consequently remain stable, with only a brief RTT fluctuation during the burst-induced path migration.

6 Discussion

(1) Design Rationale for Decoupling Cost from Scheduling. In our system design (Eq. 1 and Eq. 3), we exclude cost from runtime scheduling decisions. This is because acceleration workloads are not bandwidth-intensive, and their traffic is dominated by lightweight request-response communication, making bandwidth cost negligible compared to latency and stability constraints. Cost heterogeneity across providers is addressed at the provisioning stage through infrastructure selection and bandwidth allocation (e.g., GCP at approximately \$0.05–\$0.12/GB, compared to around \$0.01/GB for alternative providers such as Vultr and DigitalOcean). As a result, runtime scheduling focuses on latency and stability optimization, while cost efficiency is handled offline during provisioning.

(2) Design Assumptions and Operational Boundaries. Cross-cloud workloads exhibit significant temporal variability, including bursts and short-duration spikes (as shown in Fig. 3) [62]. Our system prioritizes stability within a defined capacity region. When demand exceeds capacity, overload protection mechanisms such as rate limiting or service degradation [46] are triggered and decoupled from the core scheduling logic. We assume that resources and relays operate within a trusted deployment domain with standard security mechanisms such as encrypted tunnels and access control. Controller and telemetry overhead, scalability, and comprehensive failure recovery are important for production deployment but are orthogonal to the core system design; we therefore focus on evaluating the proposed system prototype under these assumptions.

(3) Experimental Configuration and Reproducibility. Control parameters and offline configurations are engineering choices rather than universal assumptions. No single configuration fits all environments; for example, given the fully connected intercontinental topology, we recommend setting the forwarding hop limit to $H(P_{s\to d}) \le 3$. Our open-source system and the methodology described in this paper support reproduction of our testbed and provide guidance for practitioners to analyze and reconfigure these parameters for their specific settings. Broader stress testing across configurations is valuable, but off-testbed evaluation is beyond the scope of validating the core mechanisms.

7 Related Work

(1) Overlay Network. Overlay networks enhance the performance, stability, and functionality of the underlying network without altering its core architecture. Early systems like RON [9] provided resilient forwarding as a fallback to improve stability. With the rise of short-form content and interactive media platforms, overlay networks have evolved to support diverse use cases including RTC [15], video conferencing [36, 64], distributed computation [30], and P2P systems [59, 73]. Modern overlays now emphasize not only functionality but also performance, robustness, and cost efficiency.

(2) Multi-Cloud Systems. Recent systems have explored rebuilding cloud infrastructure across providers, including cross-cloud resource orchestration [68], data movement optimization [26, 63], distributed storage [37], and geo-distributed serving and workload management [34, 41, 66]. These systems primarily focus on building unified multi-cloud abstractions to coordinate computation, storage, and data movement across heterogeneous cloud providers within a single framework.

8 Conclusion

We present SkyAccel, a cross-cloud traffic acceleration system for latency-sensitive workloads (LSWs). The key insight of our design is that edge and core domains exhibit largely different dynamics, which enables an asymmetry-aware control decomposition (ACD) that separates fast-timescale stability control from slow-timescale routing optimization. This separation allows SkyAccel to achieve

stable, low-latency operation in heterogeneous multi-cloud environments while remaining cost-efficient for deployment.

References

[1] Akamai. 2026. Akamai Content Delivery Network (CDN). https://www.akamai.com/solutions/cdn

[2] Alibaba Cloud. 2026. Alibaba Cloud. https://www.alibabacloud.com/.

[3] Alibaba Cloud. 2026. Alibaba Cloud Global Acceleration. https://www.alibabacloud.com/product/ga.

[4] Alibaba Cloud. 2026. Pricing. https://www.alibabacloud.com/pricing

[5] Alibaba Group. 2026. Alibaba Cluster Trace Program. https://github.com/alibaba/clusterdata.

[6] Paul Almasan, José Suárez-Varela, Bo Wu, Shihan Xiao, Pere Barlet-Ros, and Albert Cabellos-Aparicio. 2021. Towards real-time routing optimization with deep reinforcement learning: Open challenges. In *2021 IEEE 22nd International Conference on High Performance Switching and Routing (HPSR)*. IEEE, 1–6.

[7] Amazon. 2026. Amazon Elastic Container Service (ECS). https://aws.amazon.com/ecs/

[8] Amazon. 2026. AWS Global Accelerator. https://aws.amazon.com/global-accelerator/

[9] David Andersen, Hari Balakrishnan, Frans Kaashoek, and Robert Morris. 2001. Resilient Overlay Networks. *ACM SIGOPS Operating Systems Review* (2001).

[10] Vaibhav Bajpai, Steffie Jacob Eravuchira, and Jürgen Schönwälder. 2017. Dissecting last-mile latency characteristics. *ACM SIGCOMM Computer Communication Review* 47, 5 (2017), 25–34.

[11] Rahul Bothra, Rohan Gandhi, Ranjita Bhagwan, Venkata N Padmanabhan, Rui Liang, Steve Carlson, Vinayaka Kamath, Sreangsu Acharyya, Ken Sueda, Somesh Chaturmohta, et al. 2023. Switchboard: Efficient resource management for conferencing services. In *Proceedings of the ACM SIGCOMM 2023 Conference*. 1000–1011.

[12] F Carrabs, R Cerulli, R Mansini, D Serra, and C Sorgente. 2025. Hybridizing Carousel Greedy and Kernel Search: A new approach for the maximum flow problem with conflict constraints. *European Journal of Operational Research* 324, 2 (2025), 414–435.

[13] Yingying Chen, Sourabh Jain, Vijay Kumar Adhikari, and Zhi-Li Zhang. 2011. Characterizing roles of front-end servers in end-to-end performance of dynamic content distribution. In *Proceedings of the 2011 ACM SIGCOMM conference on Internet measurement conference*. 559–568.

[14] CoreDNS. 2026. CoreDNS. https://coredns.io/.

[15] Sandesh Dhawaskar Sathyanarayana, Kyunghan Lee, Dirk Grunwald, and Sangtae Ha. 2023. Converge: QoE-driven Multipath Video Conferencing over WebRTC. In *Proceedings of the ACM SIGCOMM 2023 Conference*. New York, NY, USA, 637–653.

[16] DigitalOcean. 2026. DigitalOcean. https://www.digitalocean.com/

[17] David Eppstein. 1998. Finding the k shortest paths. *SIAM Journal on computing* 28, 2 (1998), 652–673.

[18] Yunqian Fan, Shihao Bai, Ruihao Gong, Zaijun Wang, and Rui Fan. 2026. PiLLM: Resource-Efficient LLM Inference Using Workload Prediction. In *Proceedings of the 21st European Conference on Computer Systems*. 2356–2369.

[19] Yu Gan, Yanqi Zhang, Dailun Cheng, Ankitha Shetty, Priyal Rathi, Nayan Katarki, Ariana Bruno, Justin Hu, Brian Ritchken, Brendon Jackson, et al. 2019. An open-source benchmark suite for microservices and their hardware-software implications for cloud & edge systems. In *Proceedings of the twenty-fourth international conference on architectural support for programming languages and operating systems*. 3–18.

[20] Google Cloud. 2026. Cloud Load Balancing. https://cloud.google.com/load-balancing

[21] Google Cloud. 2026. Google Cloud Platform. https://cloud.google.com/

[22] Google Cloud. 2026. Pricing. https://cloud.google.com/pricing

[23] gRPC. 2026. gRPC: A high performance, open source universal RPC framework. https://github.com/grpc/grpc

[24] Filip Idzikowski and Christian Raack. 2025. SNDlib: Survivable Network Design Library. https://sndlib.put.poznan.pl/

[25] Van Jacobson. 1988. Congestion avoidance and control. *ACM SIGCOMM computer communication review* 18, 4 (1988), 314–329.

[26] Paras Jain, Sam Kumar, Sarah Wooders, Shishir G Patil, Joseph E Gonzalez, and Ion Stoica. 2023. Skyplane: Optimizing transfer cost and throughput using {Cloud-Aware} overlays. In *20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 23)*. 1375–1389.

[27] Edwin T Jaynes. 2003. *Probability Theory: The Logic of Science*. Cambridge University Press, Cambridge, UK. Reprint with corrections.

[28] Jon Kleinberg and Eva Tardos. 2006. *Algorithm design*. Pearson Education India.

[29] Umesh Krishnaswamy, Rachee Singh, Paul Mattes, Paul-Andre C Bissonnette, Nikolaj Bjørner, Zahira Nasrin, Sonal Kothari, Prabhakar Reddy, John Abeln, Srikanth Kandula, Himanshu Raj, Luis Irun-Briz, Jamie Gaudette, and Erica Lan. 2023. OneWAN is better than two: Unifying a split WAN architecture. In *20th USENIX Symposium on Networked Systems Design and Implementation*. Boston, MA, 515–529.

[30] Dhruv Kumar, Sohaib Ahmad, Abhishek Chandra, and Ramesh K. Sitaraman. 2021. AggNet: Cost-Aware Aggregation Networks for Geo-distributed Streaming Analytics. In *IEEE/ACM Symposium on Edge Computing (SEC)*. 297–311.

[31] Adam Langley, Alistair Riddoch, Alyssa Wilk, Antonio Vicente, Charles Krasic, Dan Zhang, Fan Yang, Fedor Kouranov, Ian Swett, Janardhan Iyengar, Jeff Bailey, Jeremy Dorfman, Jim Roskind, Joanna Kulik, Patrik Westin, Raman Tenneti, Robbie Shade, Ryan Hamilton, Victor Vasiliev, Wan-Teh Chang, and Zhongyi Shi. 2017. The QUIC Transport Protocol: Design and Internet-Scale Deployment. In *Proceedings of the Conference of the ACM Special Interest Group on Data Communication*. Association for Computing Machinery, New York, NY, USA, 183–196.

[32] Nikita Lazarev, Shaojie Xiang, Neil Adit, Zhiru Zhang, and Christina Delimitrou. 2021. Dagger: efficient and fast RPCs in cloud microservices with near-memory reconfigurable NICs. In *Proceedings of the 26th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*. 36–51.

[33] Jinyang Li, Zhenyu Li, Ri Lu, Kai Xiao, Songlin Li, Jufeng Chen, Jingyu Yang, Chunli Zong, Aiyun Chen, Qinghua Wu, Chen Sun, Gareth Tyson, and Hongqiang Harry Liu. 2022. LiveNet: a low-latency video transport network for large-scale live streaming. (2022), 812–825.

[34] Zhifei Li, Tian Xia, Ziming Mao, Zihan Zhou, Ethan J Jackson, Jamison Kerney, Zhanghao Wu, Pratik Mishra, Yi Xu, Yifan Qiao, et al. 2026. SkyNomad: On Using Multi-Region Spot Instances to Minimize AI Batch Job Cost. *arXiv preprint arXiv:2601.06520* (2026).

[35] Chuanqing Lin, Gerui Lv, Fuhua Zeng, Hanlin Yang, Junwei Li, Xiaodong Li, Jingyu Yang, Yu Tian, Qinghua Wu, Zhenyu Li, et al. 2025. Oceanus: Scheduling Traffic Flows to Achieve Cost-Efficiency under Uncertainties in Large-Scale Edge CDNs. *Proceedings of the ACM on Networking* 3, CoNEXT4 (2025), 1–25.

[36] Xianshang Lin, Yunfei Ma, Junshao Zhang, Yao Cui, Jing Li, Shi Bai, Ziyue Zhang, Dennis Cai, Hongqiang Harry Liu, and Ming Zhang. 2022. GSO-simulcast: global stream orchestration in simulcast video conferencing systems. In *Proceedings of the ACM SIGCOMM Conference*. New York, NY, USA, 826–839.

[37] Shu Liu and Xiangxi Mo. 2025. SkyStore: Cost-Optimized Object Storage Across Regions and Clouds. In *Proceedings of the VLDB Endowment*, Vol. 18. 2084–2096. doi:10.14778/3734839.3734846

[38] Yilei Lu, Dongbiao He, Teng Ma, Zhe Liu, Letian Ruan, Jinlei Jiang, and Yongwei Wu. 2026. Bridging the GPU Utilization Gap: Predictive Multi-Dimensional Resource Scheduling for AI Workloads. In *Proceedings of the 21st European Conference on Computer Systems*. 383–399.

[39] Shutian Luo, Huanle Xu, Chengzhi Lu, Kejiang Ye, Guoyao Xu, Liping Zhang, Yu Ding, Jian He, and Chengzhong Xu. 2021. Characterizing Microservice Dependency and Performance: Alibaba Trace Analysis. In *Proceedings of the ACM Symposium on Cloud Computing*. 412–426.

[40] Shutian Luo, Huanle Xu, Kejiang Ye, Guoyao Xu, Liping Zhang, Guodong Yang, and Chengzhong Xu. 2022. The power of prediction: microservice auto scaling via workload learning. In *Proceedings of the 13th symposium on cloud computing*. 355–369.

[41] Ziming Mao, Tian Xia, Zhanghao Wu, Wei-Lin Chiang, Tyler Griggs, Romil Bhardwaj, Zongheng Yang, Scott Shenker, and Ion Stoica. 2025. Skyserve: Serving ai models across regions and clouds with spot instances. In *Proceedings of the Twentieth European Conference on Computer Systems*. 159–175.

[42] Michael Mitzenmacher. 2002. The power of two choices in randomized load balancing. *IEEE transactions on parallel and distributed systems* 12, 10 (2002), 1094–1104.

[43] Michael Neely. 2010. *Stochastic network optimization with application to communication and queueing systems*. Morgan & Claypool Publishers.

[44] Abhinav Pathak, Y Angela Wang, Cheng Huang, Albert Greenberg, Y Charlie Hu, Randy Kern, Jin Li, and Keith W Ross. 2010. Measuring and evaluating TCP splitting for cloud services. In *International Conference on Passive and Active Network Measurement*. Springer, 41–50.

[45] Haoran Qiu, Subho S Banerjee, Saurabh Jha, Zbigniew T Kalbarczyk, and Ravishankar K Iyer. 2020. {FIRM}: An intelligent fine-grained resource management framework for {SLO-Oriented} microservices. In *14th USENIX symposium on operating systems design and implementation (OSDI 20)*. 805–825.

[46] Harsha Sharma, Parth Thakkar, Sagar Bharadwaj, Ranjita Bhagwan, Venkata N Padmanabhan, Yogesh Bansal, Vijay Kumar, and Kathleen Voelbel. 2022. Optimizing network provisioning through cooperation. In *19th USENIX Symposium on Networked Systems Design and Implementation (NSDI 22)*. 1179–1194.

[47] Abhishek Sinha and Eytan Modiano. 2018. Network utility maximization with heterogeneous traffic flows. In *2018 16th International Symposium on Modeling and Optimization in Mobile, Ad Hoc, and Wireless Networks (WiOpt)*. IEEE, 1–8.

[48] Smux. 2026. Smux. https://github.com/xtaci/smux.

[49] Yu Tian, Zhenyu Li, Matthew Yang Liu, Jian Mao, Gareth Tyson, and Gaogang Xie. 2024. Cost-Saving Streaming: Unlocking the Potential of Alternative Edge Node Resources. In *Proceedings of the 2024 ACM on Internet Measurement Conference*. 580–587.

[50] Yu Tian, Zhenyu Li, Matthew Yang Liu, Qinghua Wu, Zhaoxue Zhong, Ao Li, Jiaxing Zhang, Gerui Lv, Chuanqing Lin, Xi Wang, et al. 2025. Cost-efficient Request Mapping for Large-scale Live Streaming Services. *Proceedings of the ACM on Networking* 3, CoNEXT4 (2025), 1–26.

[51] Yu Tian, Gerui Lv, Qinghua Wu, Ruili Fang, Yajie Peng, Zhichen Xue, Rui Han, Chuanqing Lin, Xiaofei Pang, Ri Lu, et al. 2026. RLive: Robust Delivery System for Scaling Live Streaming Services. In *Proceedings of the 21st European Conference on Computer Systems*. 1604–1620.

[52] TikTok. 2026. TikTok. https://www.tiktok.com/.

[53] Alexandru Uta, Alexandru Custura, Dmitry Duplyakin, Ivo Jimenez, Jan Rellermeyer, Carlos Maltzahn, Robert Ricci, and Alexandru Iosup. 2020. Is big data performance reproducible in modern cloud networks?. In *17th USENIX symposium on networked systems design and implementation (NSDI 20)*. 513–527.

[54] Vultr. 2026. Vultr. https://www.vultr.com/.

[55] Haiping Wang, Wanxin Shi, Sandesh Dhawaskar Sathyanarayana, Shu Shi, Feng Qian, Yinghao Yu, La Zuo, Hebin Yu, Ruixiao Zhang, Ruoshi Sun, et al. 2026. Medley: Optimizing Midgress Bandwidth for Commercial Live Streaming {CDNs}. In *23rd USENIX Symposium on Networked Systems Design and Implementation (NSDI 26)*. 2553–2568.

[56] Kangjin Wang, Ying Li, Cheng Wang, Tong Jia, Kingsum Chow, Yang Wen, Yaoyong Dou, Guoyao Xu, Chuanjia Hou, Jie Yao, et al. 2022. Characterizing job microarchitectural profiles at scale: Dataset and analysis. In *Proceedings of the 51st international conference on parallel processing*. 1–11.

[57] Liying Wang, Jing Liu, Yuhan Zhou, Chengke Wang, Mingming Lu, Qingyue Li, Song Geng, Linsen Wang, Kaida Hu, Haoyuan Huang, et al. 2026. {HCDN}: Coordinated Stream Scheduling for {Cost-Effective} Live Video Delivery. In *23rd USENIX Symposium on Networked Systems Design and Implementation (NSDI 26)*. 2603–2617.

[58] webrtc. 2026. WebRTC: Real-Time Communication for the Web. https://webrtc.org/

[59] Dehui Wei, Jiao Zhang, Haozhe Li, Rui Han, Zhichen Xue, Yajie Peng, Xiaofei Pang, Yan Ma, and Jialin Li. 2026. {HyperEdge}: An Edge {CDN} Infrastructure for Cost Efficient Video Streaming. In *23rd USENIX Symposium on Networked Systems Design and Implementation (NSDI 26)*. 2635–2650.

[60] Qizhen Weng, Wencong Xiao, Yinghao Yu, Wei Wang, Cheng Wang, Jian He, Yong Li, Liping Zhang, Wei Lin, and Yu Ding. 2022. MLaaS in the Wild: Workload Analysis and Scheduling in Large-Scale Heterogeneous GPU Clusters. In *19th {USENIX} Symposium on Networked Systems Design and Implementation ({NSDI} 22)*.

[61] Qizhen Weng, Lingyun Yang, Yinghao Yu, Wei Wang, Xiaochuan Tang, Guodong Yang, and Liping Zhang. 2023. Beware of Fragmentation: Scheduling GPU-Sharing Workloads with Fragmentation Gradient Descent. In *2023 USENIX Annual Technical Conference (USENIX ATC '23)*. USENIX Association. https://www.usenix.org/conference/atc23/presentation/weng

[62] Rajini Wijayawardana and Andrew A Chien. 2025. Scheduling Cloud VMs on Variable Capacity Datacenters. In *Proceedings of the 2025 ACM Symposium on Cloud Computing*. 507–520.

[63] Sarah Wooders, Shu Liu, Paras Jain, Xiangxi Mo, Joseph E Gonzalez, Vincent Liu, and Ion Stoica. 2024. Cloudcast:{High-Throughput},{Cost-Aware} overlay multicast in the cloud. In *21st USENIX Symposium on Networked Systems Design and Implementation (NSDI 24)*. 281–296.

[64] Bingyang Wu, Kun Qian, Bo Li, Yunfei Ma, Qi Zhang, Zhigang Jiang, Jiayu Zhao, Dennis Cai, Ennan Zhai, Xuanzhe Liu, and Xin Jin. 2023. XRON: A Hybrid Elastic Cloud Overlay Network for Video Conferencing at Planetary Scale. In *Proceedings of the ACM SIGCOMM Conference*. New York, NY, USA, 696–709.

[65] Zhanghao Wu, Wei-Lin Chiang, Ziming Mao, Zongheng Yang, Eric Friedman, Scott Shenker, and Ion Stoica. 2024. Can't be late: optimizing spot instance savings under deadlines. In *21st USENIX Symposium on Networked Systems Design and Implementation (NSDI 24)*. 185–203.

[66] Tian Xia, Ziming Mao, Jamison Kerney, Ethan J. Jackson, Zhifei Li, Jiarong Xing, Scott Shenker, and Ion Stoica. 2026. SkyWalker: A Locality-Aware Cross-Region Load Balancer for LLM Inference. In *Proceedings of the 21st European Conference on Computer Systems*. Association for Computing Machinery, New York, NY, USA, 1349–1364.

[67] Lingyun Yang, Yongchen Wang, Yinghao Yu, Qizhen Weng, Jianbo Dong, Kan Liu, Chi Zhang, Yanyi Zi, Hao Li, Zechao Zhang, Nan Wang, Yu Dong, Menglei Zheng, Lanlan Xi, Xiaowei Lu, Liang Ye, Guodong Yang, Binzhang Fu, Tao Lan, Liping Zhang, Lin Qu, and Wei Wang. 2025. GPU-Disaggregated Serving for Deep Learning Recommendation Models at Scale. In *22nd USENIX Symposium on Networked Systems Design and Implementation (NSDI 25) (USENIX NSDI '25)*. USENIX Association. https://www.usenix.org/conference/nsdi25/presentation/yang

[68] Zongheng Yang, Zhanghao Wu, Michael Luo, Wei-Lin Chiang, Romil Bhardwaj, Woosuk Kwon, Siyuan Zhuang, et al. 2023. SkyPilot: An intercloud broker for sky computing. In *20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 23)*. 437–455.

[69] Chengzhi Lu Chengzhong Xu Yanying Lin, Shijie Peng and Kejiang Ye. 2026. FlexPipe: Adapting Dynamic LLM Serving Through Inflight Pipeline Refactoring in Fragmented Serverless Clusters. In *European Conference on Computer Systems (EuroSys '26)*. 1–17. doi:10.1145/3767295.3769316

[70] Shutian Luo Hong Xu Haiying Shen Chong Ma Min Shen Le Chen Chengzhong Xu Lin Qu Kejiang Ye Yanying Lin, Shuaipeng Wu. 2025. Understanding Diffusion Model Serving in Production: A Top-Down Analysis of Workload, Scheduling, and Resource Efficiency. In *ACM Symposium on Cloud Computing (SoCC '25)*.

[71] YouTube. 2026. YouTube. https://www.youtube.com/.

[72] Jinkun Zhang, Yuezhou Liu, and Edmund Yeh. 2025. Delay-optimal Congestion-aware Routing and Computation Offloading in Arbitrary Network. *arXiv preprint arXiv:2506.13626* (2025).

[73] Rui-Xiao Zhang, Haiping Wang, Shu Shi, Xiaofei Pang, Yajie Peng, Zhichen Xue, and Jiangchuan Liu. 2024. Enhancing Resource Management of the World's Largest PCDN System for On-Demand Video Streaming. In *USENIX Annual Technical Conference (USENIX ATC)*. Santa Clara, CA, 951–965.

[74] Wei Zhang, Tong Meng, Xianhua Zeng, Wei Yang, Changqing Yan, Chao Li, Chenguang Li, Feng Qian, Junfeng Yang, Lei Zhang, et al. 2025. Harnessing WebRTC for Large-Scale Live Streaming. In *Proceedings of the ACM SIGCOMM 2025 Conference*. 1199–1212.

[75] Yuxuan Zhao, Dmitry Duplyakin, Robert Ricci, and Alexandru Uta. 2021. Cloud performance variability prediction. In *Companion of the ACM/SPEC International Conference on Performance Engineering*. 35–40.

Algorithm 1: Carousel Greedy Algorithm for the MFPC

Input: Original graph $G = (V, A)$; source node s; sink node t; α and β are control parameters.
Output: Solution S^*.

1 **Init:** $S \leftarrow \langle\rangle$; $G_f \leftarrow$ residual graph of G;
2 Greedily select P $(s \rightarrow t)$ satisfying admission and latency; add to S, update flow and G_f until no more feasible paths exist;
3 $\mathcal{P} \leftarrow \{P : P \in S\}$; $\tau_P \leftarrow 1, \forall P \in S$;
4 $S^* \leftarrow S$; $z^* \leftarrow z(S)$;
5 $S^C \leftarrow$ Remove the $\beta \cdot |S|$ newly selected paths from S;
6 **for** $k \leftarrow 1$ ***to*** $\alpha|S|$ **do**
7 Remove the oldest path P^O from S^C;
8 Update the residual network G_f;
9 $\tau_{\max} \leftarrow \max_{P \in \mathcal{P}} \tau_P$; `// highest frequency`
10 Let $P_{\tau_{\max}}$ be the oldest path with frequency $\tau_{\max}$; Zero residuals on first arc of P^O and all arcs of $P_{\tau_{\max}}$;
11 Find P in G_f s.t. admission & latency hold;
12 **while** *P is not empty* **do**
13 **if** $P \in \mathcal{P}$ **then**
14 $\tau_P \leftarrow \tau_P + 1$
15 **else**
16 $\mathcal{P} \leftarrow \mathcal{P} \cup \{P\}$; $\tau_P \leftarrow 1$
17 Add P to S^C, compute the flow increment for path P, and update the residual network G_f;
18 Find path P in G_f satisfies both admission limit and latency constraints;
19 **If** $z^* < z(S^C)$ **then** $S^* \leftarrow S^C$; $z^* \leftarrow z(S^C)$;
20 **return** S^*

A Carousel Greedy Approach for the MFPC

This section describes our implementation of the Carousel Greedy algorithm proposed in [12]. For brevity, we omit the full pseudocode and provide a concise summary of our implementation in the appendix, highlighting the aspects relevant to evaluating its effectiveness and efficiency in our setting.

Algorithm 1 initializes the path set S and residual network G_f, then iteratively selects feasible paths P from s to t, adds them to S, and updates the flow. Once no more paths are found, each $P \in S$ is added to the global set $\mathcal{P}$ with frequency $\tau_P = 1$. The initial solution is stored as S^* with objective value z^* (lines 1–4). The Carousel process then begins by removing the most recently selected $\beta \cdot |S|$ paths from S, yielding the carousel working set S^C, followed by $\alpha \cdot |S|$ iterations to search for an improved solution (lines 5–6). In each iteration, the oldest path is removed from S^C, and the most frequently selected path is identified, with ties broken by preferring earlier-generated candidates (lines 7–10). To promote path diversity, the first arc of the oldest path and all arcs of the most frequent path $P_{\tau_{\max}}$ are banned in the residual graph by zeroing their residual capacity. A new feasible path P is then searched (lines 11–12). If P is non-empty, the algorithm continues to search for additional valid paths under the updated constraints (lines 13–17). If the objective value improves, the best solution S^* and its value z^* are updated (line 18). After all iterations, the optimal solution found, S^*, is returned (line 20). The overall logic is illustrated in Fig. 13.

The control parameters α and β are configured according to the network topology and connectivity. For the `native` and `cost266` topologies evaluated in our experiments, we set $\alpha = 2.0$ and $\beta = 0.5$, meaning that the Carousel process performs $2|S|$ iterations and initially removes the most recently selected $0.5|S|$ paths from the solution.

The Carousel Greedy algorithm balances solution quality and computational efficiency through iterative path selection and residual-network updates. By temporarily banning the first arc of the oldest path and all arcs of the most frequently selected path, it encourages path diversity and explores alternative feasible solutions. The algorithm maintains the best solution found throughout the search and updates it whenever an improved solution is obtained. This iterative strategy provides an efficient heuristic for MFPC, particularly for large-scale instances where exact optimization may be computationally prohibitive.

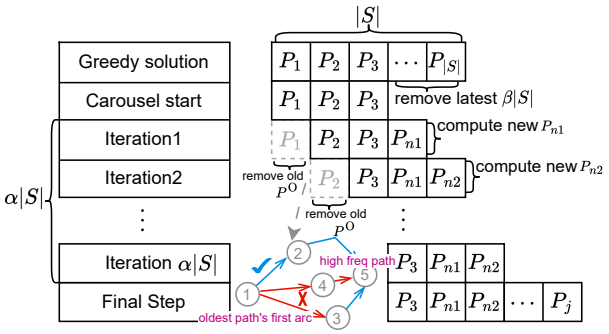


Figure 13: The logic diagram of the Carousel Greedy, which consists of three parts: the algorithm flow (left), the evolution of the path set (right), and the topology-based arc filtering (bottom). The left panel shows the algorithm flow: starting from a greedy initial solution, it runs for $\alpha|S|$ iterations, each performing two steps—removing old paths and computing a new path—before reaching the final step. The right panel illustrates how the path set $S = \{P_1, P_2, \ldots, P_{|S|}\}$ evolves across iterations: in each round, a number of old paths are removed (e.g., the latest $\beta|S|$ paths or a designated old path P_i^{old}), and a new path (e.g., P_{n1}, P_{n2}) is computed and added. The bottom topology explains the basis for removal and computation in each iteration: red segments denote arcs excluded from consideration, namely the first arc of the oldest path and all arcs of the most frequently selected path; blue segments denote the remaining available arcs. Arrows ①–⑤ depict this filtering process—identifying and removing the two types of red arcs, then computing a new path over the remaining blue arcs.